

PLANO DE AÇÃO: Replicação de Componentes da Fábrica Agêntica

Data: 2026-08-23

Contexto: Análise completa do projeto para extrair arquitetura, scripts, skills, MCPs, hooks, specs e rules reutilizáveis em novos projetos.

Objetivo: Permitir que projetos novos nasçam já configurados com toda a infraestrutura técnica e metodologia aplicada.

EXECUTIVE SUMMARY

O projeto **Fábrica Agêntica de Publicações** é um sistema modular e extensível baseado em:

- **50+ Skills especializadas** (economia de tokens + redação + compilação)
- **19 Comandos orquestrador** (P&D → Manufatura → Review → PDF)
- **7 Subagentes paralelos** (pesquisador, redators, revisor, ilustrador)
- **70+ Scripts determinísticos** (indexação, gates, compilação, zero-LLM)
- **4 MCPs** (estado, escrita, PDF, inteligência de grafo)
- **Templates reutilizáveis** (Markdown EITA-V2, Typst ABNT, HTML responsivo)
- **V5 Sistema de Coleções** (cascata livro → artigo → e-book → playbook → lead magnet → deck → e-mails)
- **Portabilidade Multi-IDE** (Claude Code, Cursor, OpenCode, Codebuff, VSCode)
- **Token Economy** (caveman, headroom, lean-ctx, RTK — economiza até 70%)

Para novo projeto: copiar estrutura, rodar `setup-links.ps1`, adaptar `CLAUDE.md` (tema), customizar `tipos_obra.py` (tipos novos), executar gates. **Projeto nasce pronto.**

FASE 1: NÚCLEO DE GOVERNANÇA (blueprint estrutural)

1.1 — Hierarquia de Configuração (3 camadas)

Projeto novo

└─ .claude/CLAUDE.md	[raiz: regras globais, squad, fluxo]
└─ .claude/settings.json	[hooks, MCPs, permissions]
└─ .claude/RTK.md	[token economy local]
└─ CLAUDE.md (link → .claude/)	[visibilidade em IDE]
└─ .claude.yaml / opencode.json	[suporte Cursor/OpenCode]

O que replicar:

- **Frontmatter em CLAUDE.md:**

```
---
description: Regras, squad e fluxo da [Fábrica/Sistema] – orquestrador para
qualquer agente neste diretório.
alwaysApply: true
---
```

- **Estrutura de seções numeradas:**

- Seção 0: Economia Severa de Tokens (PRIORIDADE MÁXIMA)
- Seção 1: Regras Globais (R1–R17)
- Seção 2: Squad (F1–F3, skills + scripts + gates)
- Seção 3: MCPs
- Seção 4: Templates
- Seção 5: Fluxo Operacional (5–9 fases)
- Seção 6: Portabilidade Multi-IDE (junctions, hardlinks, setup)

- Seção 7: RTK SCRATCHPAD (aprendizados de sessões anteriores)
- **Glossário de Regras (R1–R17):**
 - R1: PT-BR estrito em toda comunicação
 - R2: Silenciamento estético (sem preâmbulos em artefatos)
 - R3: Autonomia (após tema definido, sistema roda 100%)
 - R4: Auto-correção (desvios internos, sem output de falha)
 - R5: Capa 2D Plano (Livro/E-book) vs ABNT (TCC/Artigo)
 - R6: Modelo LLM livre (model: inherit)
 - R16: Pós-implementação (testes 100%, nunca commit vermelho)
 - R17: CAMPANHA e MÁQUINA opcionais (respeitam config_obra.json)
- **Squad Declarativo:**
 - F1 (Pesquisa): pesquisador → arquiteto
 - F2 (Manufatura): estrategista → redator-* → subagentes paralelos (pool lotes 4)
 - F2.5 (Peer Review): auditar-obra.py → revisor-tecnico (apenas defeituosas)
 - F3 (Compilação): compilador-abnt → compilar-para-pdf.py
 - F4/V5 (Coleção): criar-playbook → derivadas paralelas → sincronizar → empacotar
- **Tipos de Obra (V5):**
 - Tabela: tipo | natureza | custo LLM | spec | comando | produtor
 - Registro em scripts/tipos_obra.py (uma entrada por tipo)
 - Matriz de derivação (cascata): livro → artigo, e-book, playbook, lead magnet, deck, e-mails

1.2 — Hooks de Pós-Implementação (settings.json)

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "code-review-graph update --skip-flows --repo \"$(git rev-parse --show-toplevel 2>/dev/null)\""
          },
          {
            "type": "command",
            "command": "python scripts/atualizar-documentacao.py --se-sujo --silencioso"
          }
        ]
      }
    ],
    "SessionStart": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "code-review-graph status --repo \"$(git rev-parse --show-toplevel 2>/dev/null)\""
          }
        ]
      }
    ]
  }
}
```

```

    ]
  }
]
}
}

```

Adaptar: substituir paths absolutos de Windows por `git rev-parse --show-toplevel`.

1.3 — Junctions (Portabilidade Multi-IDE)

```

# scripts/setup-links.ps1
New-Item -ItemType SymbolicLink -Path "agentic" -Target ".claude" -Force
New-Item -ItemType SymbolicLink -Path ".agents/agents" -Target ".claude/agents" -
Force
New-Item -ItemType SymbolicLink -Path ".agents/commands" -Target ".claude/commands" -
Force
New-Item -ItemType HardLink -Path "CLAUDE.md" -Target ".claude/CLAUDE.md" -Force
Copy-Item "scripts/hooks/pre-commit" ".git/hooks/pre-commit" -Force

```

Benefício: um único CLAUDE.md suporta: - Claude Code: `.claude/` - Cursor: `.cursor/rules/` - OpenCode: `.opencode/` - Codebuff: `.agents/` - VSCode: `.vscode/mcp.json` (gerado por script)

FASE 2: CATÁLOGO DE SKILLS (50+ reutilizáveis)

2.1 — Mapeamento de Skills por Camada

[FUNDAÇÃO — Economia de Tokens]

- └ caveman (pensamento telegráfico: 3–5 linhas)
- └ headroom (compress logs >7 linhas: 3 top + 4 bottom)
- └ lean-ctx (grep antes de read; limitar linha)
- └ rtk-memory (rastreamento de economias em sessão)
- └ pre-flight-check (validação pré-launch, evita loops)
- └ aplicar-token-economy (configurar estratégia de compressão)

[FÁBRICA — Redação & Compilação]

- └ pesquisador (dossiê + referências; F1 Nó 1)
- └ arquiteto (sumário macro; F1 Nó 2)
- └ estrategista (draft capítulo; F2 Nó 3)
- └ redator-eita (framework EITA-V2; F2 Nó 4)
- └ redator-academico (ABNT + IMRaD; para TCC/Artigo)
- └ redator-ebook (compressão; herança livro)
- └ revisor-tecnico (peer review; F2.5)
- └ compilador-abnt (merge + formatação; F3)
- └ compilador-artigo (gates acadêmicas; F3)
- └ compilador-mega-livro (orquestração final; F3)

[GATES — Validação Determinística]

- └ validar-referencias.py (URL/DOI reais; reprova 4xx/DNS)
- └ validar-metricas.py (≥ 1 métrica com valor+unidade+citação)
- └ validar-escala.py (contorno em "Aplica" existe)
- └ validar-afirmacoes.py (dado factual tem [N])
- └ validar-fontes.py ($\geq 70\%$ A+B)
- └ validar-comandos-cli.py (smoke test real; opt-in)
- └ auditar-obra.py (encadeia todos; --estrito)

[DERIVADAS — Coleção V5]

- └ criar-playbook (cards práticos; zero LLM)
- └ criar-lead-magnet (A4 + CTA; HTML→Chromium)
- └ criar-deck (16:9 navegável; 2 formatos)

- └ criar-emails (sequência marketing; moldes)
- └ campanha (conteúdo marketing; estrutura+artes)
- └ criar-maquina-vendas (Next.js + FastAPI; deploy + segurança)

[INFRAESTRUTURA]

- └ code-review (review de PR; adversarial)
- └ debug-issue (diagnostic; root cause)
- └ refactor-safely (reescrita com testes)
- └ atualizar-documentacao (sync manual <-> CLAUDE.md)
- └ explore-codebase (busca rápida; Explore agent)

[TERCEIRAS – Cloudflare/Vercel/etc]

- └ [20+ skills de plataforma]

2.2 – Estrutura Canônica de Skill

name: nome-skill

description: Uma frase descrevendo o resultado final (aparece em /help)

Skill_NomeCapitalizado

Você é o especialista em [domínio] na [Fábrica/Sistema].

Contexto & Objetivo

[Onde estamos no fluxo, o que este skill deve entregar]

Regras

- REGRA X do CLAUDE.md (R1 PT-BR, R2 silenciamento, R4 auto-correção)
- Restrições específicas deste skill

Padrão de Fluxo

Passo 1 – [Preparação]

[Validações, criação de pastas, leitura de config]

Passo 2 – [Processamento]

[Lógica do skill, invocações de scripts/subagentes]

Passo 3 – [Validação & Entrega]

[Gates, testes, gravação de relatório]

Checklist de Entrega

- [] Output em `output/<slug>/...`
- [] `relatorio-<skill>.json` gravado
- [] Testes executados (auditar-obra.py)
- [] Commit + push feito

Reutilização: copiar estrutura, adaptar domínio e padrão de fluxo.

FASE 3: SISTEMA DE COMANDOS (19 orquestradores)

3.1 – Padrão de Comando

description: Resumo do fluxo (e.g., "Inicia a produção autônoma de um livro técnico")

Situação Inicial

[Contexto: trigger, o que o operador está fazendo]

REQUISITOS CONTRATUAIS – NÃO NEGOCIÁVEIS

#	Requisito	Especificação
R1	16+ capítulos	Mínimo 16 capítulos no sumário macro
R2	70+ páginas	Mínimo ~175.000 caracteres em `livro_final.md`

...

Passo 0 – Preparação

[Criar pastas, validar config_obra.json, registrar em MCP db_state]

Passo 1 – Fase 1 (P&D e Arquitetura)

[Invoque subagentes, execute scripts, persist dados]

Passo 2 – Fase 2 (Manufatura em Lotes)

[Pool de capítulos, dispatch paralelo, aguarde todos]

Passo 3 – Fase 2.5 (Peer Review)

[Auditoria gates, correção técnica, re-validação]

Passo 4 – Fase 3 (Compilação + PDF)

[Merge de capítulos, ABNT, Pandoc→Typst]

Passo 5 – Fase 4/V5 (Coleção & Entrega)

[Derivadas paralelas, sincronizar, empacotar]

Passo 6 – Relatório Telegráfico (REGRA 2)

[Resumo de entregas, commits, próximos passos]

3.2 – Fluxo Canônico de Orquestração

entrada (tema)

↓

[Fase 1] pesquisador → minerar-fontes-academicas.py → indexar-dossie.py → arquiteto

↓

[Fase 2] pool-capitulos.py --plano --lote 4

↓

para cada lote:

- [subagente-redator-capitulo × 4] em paralelo
- aguarde todos
- se erro: tentativa 3x com backoff exponencial
- se ainda falha: escalar para revisor-tecnico

↓

[Fase 2.5] auditar-obra.py --estrito

(encadeia: validar-referencias, validar-metricas, validar-escala, validar-afirmacoes, validar-fontes, validar-codigo)

↓

se falhas detectadas:

- revisor-tecnico corrige apenas defeituosas
- re-run gates até sucesso

↓

[Fase 3] compilador-abnt → compilar-para-pdf.py

↓

[Fase 4/V5] criar-playbook

- [criar-lead-magnet, criar-deck, criar-emails] em paralelo

```

→ colecao.py --sincronizar
→ empacotar-colecao.py
↓
validar-artefatos.py --todos --estrito
↓
commit + push

```

Reutilização: substituir nomes de scripts/skills, manter fluxo e barreiras.

FASE 4: SUBAGENTES ESPECIALIZADOS (7, 100% paralelos)

4.1 — Registro de Subagentes

.claude/agents/	
├─ subagente-pesquisador.md	(varredura web + dossiê RAG)
├─ subagente-redator-capitulo.md	(draft + EITA + diagrama + CI)
├─ subagente-redator-secao-tcc.md	(análogo redator-capitulo, para TCC)
├─ subagente-redator-artigo.md	(compressão livro → artigo IMRaD)
├─ subagente-adaptador-ebook.md	(adapt tom, gera EPUB)
├─ subagente-revisor-tecnico.md	(peer review, confere 1 fonte/cap)
└─ subagente-ilustrador.md	(HTML+CSS+Playwright → PNG 2D flat)

4.2 — Padrão de Subagente

```

name: subagente-nome
description: Tarefa atomizada para execução paralela

```

Subagente_Nome

Você é o especialista em [domínio] – executa tarefa atomizada para a esteira.

Input Esperado

```

- `config_obra.json` (invariante)
- `output/<slug>/pesquisa/dossie.json` (RAG indexado)
- `lote.json` com IDs de capítulos/seções

```

Output Obrigatório

```

```json
{
 "status": "sucesso|falha|incompleto",
 "capitulos": [
 {
 "id": "cap_01",
 "conteudo": "...",
 "metricas": {"palavras": 5000, "citacoes": 15, "diagramas": 1},
 "tempo_minutos": 8
 }
],
 "relatorio": "sumário de diffs, mudanças estruturais, gaps detectados",
 "tempo_total_minutos": 45
}

```

### Retentativa (máx 3x com backoff exponencial)

- Falha 1ª: retry em 5s
- Falha 2ª: retry em 30s

- Falha 3ª: escalar para revisor-tecnico com `flag manual_fix_required=true`

### ### 4.3 – Pool de Capítulos (Orquestração)

```

python
scripts/pool-capitulos.py
def despacher_lotes(slug, lote_size=4, max_retries=3):
 """
 Divide capítulos em lotes, despacha subagentes em paralelo,
 aguarda todos (barreira), escalata defeituosos.
 """
 capitulos = carrega_plano(slug)
 lotes = [capitulos[i:i+lote_size] for i in range(0, len(capitulos), lote_size)]

 for idx, lote in enumerate(lotes):
 print(f"🚀 Lote {idx+1}: capítulos {lote[0].numero}--{lote[-1].numero}")

 # Paralelo
 tasks = [
 agent(
 f"Rediga capítulo {cap.numero}: {cap.titulo}",
 label=f"cap_{cap.numero:02d}",
 schema=CAPITULO_SCHEMA
)
 for cap in lote
]
 resultados = await all(tasks)

 # Barreira: aguarda TODOS antes do lote seguinte
 falhas = [r for r in resultados if r['status'] != 'sucesso']

 if falhas:
 print(f"⚠️ {len(falhas)} capítulos com problema, escalando para revisor")
 escalar_para_revisor(falhas, retry_count=1)

 print(f"✓ Lote {idx+1} completo")

```

## FASE 5: SCRIPTS DETERMINÍSTICOS (70+, zero LLM)

### 5.1 – Categorias de Scripts

**[A] Coleta & Indexação** - `minerar-fontes-academicas.py` — OpenAlex/Crossref/arXiv/SciELO (APIs abertas, zero LLM) - `indexar-dossie.py` — RAG vetorizado para consulta pelos redatores

**[B] Estruturação** - `tipos_obra.py` — registro declarativo de tipos (uma entrada = novo tipo) - `parametros_obra.py` — `config_obra.json` + metadados - `secoes_eita.py` — parser de headers EITA (7 seções) - `colecacao.py` — sincronização de manifestos, hub de coleção - `fatiar-obra.py` — splitter de capítulos para pool

**[C] Gates de Conteúdo** (`auditar-obra.py` — estrito encadeia todos) - `validar-referencias.py` — URL/DOI reais (4xx reprova) - `validar-metricas.py` — ≥1 métrica com valor+unidade+citação - `validar-escala.py` — contorno em “Aplica” existe - `validar-afirmacoes.py` — dado factual tem [N] - `validar-fontes.py` — ≥70% referências classe A+B - `validar-comandos-cli.py` — smoke test real (opt-in) - `validar-codigo.py` --executar — testa python/js/bash/playbook - `auditar-obra.py` — orquestrador: encadeia gates por tipo

**[D] Compilação** - `compilar-para-pdf.py` — Pandoc → Typst → `typst compile` - `gerar-epub.py` — E-book final - `gerar-pptx.py` — Deck em PPTX - `gerar-capa.py` — capa gráfica (2D plano ou ABNT) - `renderizar-diagramas.py` — Mermaid → SVG + validate

**[E] Tipos Derivados V5** (zero/extracao — determinísticos) - `gerar-lead-magnet.py` — A4 + CTA (HTML→Chromium) - `gerar-lead-magnet-pdf.py` — PDF via Playwright - `gerar-deck.py` — esqueleto 16:9 - `gerar-deck-html.py` — slides HTML navegáveis - `extrair-passos-praticos.py` — cards do playbook - `gerar-sequencia-emails.py` — moldes de copy - `criar-campanha.py` — estrutura + artes (0 token)

**[F] Validação de Entrega** - `validar-artefatos.py --todos --estrito` — cada arquivo abre - `validar-playbook.py`, `validar-lead-magnet.py`, `validar-deck.py`, `validar-emails.py` — gates específicas - `validar-abnt-tcc.py` — NBR 14724

**[G] Utilitários & Transformação** - `nomes_curtos.py` — V5.1: encurtar paths (MAX\_PATH=260 Windows) - `empacotar-colecao.py` — .zip com manifesto + LICENCA.txt + LEIA-ME.md - `migrar-slug.py`, `migrar-derivados.py` — refactoring em massa - `corrigir-nomenclatura.py` — renomear arquivos legados - `task_router.py` — dispatcher automático de tarefas - `atualizar-documentacao.py` — sincronizar manual

**[H] Operacional** - `calcular-gastos-sessao` (skill) — token economy reporting - `gerar-relatorio-sessao.py` — MD+PDF com contexto/bugs/validações - `token-guard.py` — vigilância de TPM/RPM - `verificar-rate-limits.py` — Cloudflare/OpenAI quotas

## 5.2 — Padrão Canônico de Script Python

```
#!/usr/bin/env python3
"""
```

```
<modulo>: <descricao-do-que-faz>
```

Uso:

```
python scripts/meu-script.py <slug> --opcao valor
python scripts/meu-script.py --help
```

Entrada: ``output/<slug>/pesquisa/dossie.json`` (ou `config_obra.json`)

Saída: ``output/<slug>/relatorio-meu-script.json``

Garantias:

- Idempotente (rodar 2x = mesmo resultado)
- Sem side effects fora de ``output/``
- Relatorio JSON sempre gravado (mesmo em erro)
- Exit code 0 = sucesso, >0 = falha (pre-commit hook respeita)

```
import argparse
import json
import sys
from pathlib import Path
```

```
def console_utf8():
 """Previne UnicodeEncodeError em Windows (cp1252 vs UTF-8)."""
 for fluxo in (sys.stdout, sys.stderr):
 try:
 fluxo.reconfigure(encoding="utf-8", errors="replace")
 except (AttributeError, OSError):
 pass
```

```

def main():
 parser = argparse.ArgumentParser(description=__doc__)
 parser.add_argument("slug", help="slug-da-obra")
 parser.add_argument("--opcao", default="valor_padrao", help="Descrição")
 args = parser.parse_args()

 relatorio = {
 "status": "falha",
 "erros": [],
 "alertas": [],
 "dados": {}
 }

 try:
 # Sua lógica aqui
 relatorio["status"] = "sucesso"
 except Exception as e:
 relatorio["erros"].append(str(e))
 return 1
 finally:
 caminho_rel = Path(f"output/{args.slug}/relatorio-meu-script.json")
 caminho_rel.parent.mkdir(parents=True, exist_ok=True)
 caminho_rel.write_text(json.dumps(relatorio, indent=2, ensure_ascii=False))

 return 0

if __name__ == "__main__":
 console_utf8()
 sys.exit(main())

```

**Reutilização:** template de argparse, console\_utf8(), estrutura de relatorio JSON, exit code, idempotência.

## FASE 6: MCPs (Model Context Protocols) — 4 + extensíveis

### 6.1 — Registro de MCPs em .mcp.json

```

{
 "mcpServers": {
 "db_state": {
 "command": "node",
 "args": [
 "<path>/mcp-server-sqlite/dist/index.js",
 "<path>/data/estado_fabrica.db"
]
 },
 "file_writer": {
 "command": "node",
 "args": [
 "<path>/server-filesystem/dist/index.js",
 "<project-root>"
]
 },
 "pdf_gen": {
 "command": "node",
 "args": ["<path>/pdf-gen-server/index.js"]
 }
 }
}

```

```

 "code-review-graph": {
 "command": "uvx",
 "args": ["code-review-graph", "serve"],
 "cwd": "<project-root>",
 "type": "stdio"
 }
 }
}

```

## 6.2 – Função de cada MCP

MCP	Função	Quando Usar
<b>db_state</b>	SQLite com estado da esteira (fases, capítulos, status)	Registrar progresso, queries de auditoria
<b>file_writer</b>	Escrita segura de Markdown/JSON com validação	Gerar capítulos, config, manifestos
<b>pdf_gen</b>	Compilação PDF (fallback CloudConvert)	Método principal: Pandoc→Typst via <code>compile-par-pdf.py</code>
<b>code-review-graph</b>	Inteligência estrutural do grafo (queries, impacto)	Busca eficiente, impact analysis, compliance

## 6.3 – Padrão para Novo MCP

```

// .claude/mcp-servers/meu-mcp/index.js
const { Server } = require("@modelcontextprotocol/sdk/server/stdio");

const server = new Server({
 name: "meu-mcp",
 version: "1.0.0"
});

server.setRequestHandler("resources/list", async () => ({
 resources: [
 {
 uri: "meu://recurso",
 name: "Meu Recurso",
 mimeType: "application/json"
 }
]
}));

server.setRequestHandler("resources/read", async (request) => {
 if (request.params.uri === "meu://recurso") {
 return { contents: [{ text: JSON.stringify({...}), mimeType: "application/json" }] };
 }
 throw new Error("Recurso não encontrado");
});

server.connect(process.stdin, process.stdout);

```

## FASE 7: TEMPLATES REUTILIZÁVEIS

### 7.1 — Markdown (Estrutura)

```
templates/
├─ template_eita.md # 7 seções: Intro, Explica, Ilustra, Técnica, Aplica,
Conclusão, Refs
├─ template_playbook.typ # Cards de passo-a-passo com ①②③
├─ template_lead_magnet.html # A4 responsivo + CTA bottom
└─ template_deck.html # 16:9 navegável + PDF via Chromium
```

#### Exemplo: template\_eita.md

```
Capítulo <N>: <Título>

1. Introdução
<!-- Contextualização, relevância, o que será abordado. Tom acessível. Máx 2
parágrafos. -->

2. Explica
<!-- Teoria fundamental, definições, causa raiz. Citações [N] obrigatórias. -->

3. Ilustra
<!-- Analogia + 1 diagrama ``mermaid OBRIGATÓRIO -->

4. Técnica
<!-- Código com linguagem declarada. Mínimo 60% do capítulo. [N] obrigatórias. -->

5. Aplica
<!-- Caso de uso real. Contorno/limites/escala OBRIGATÓRIOS. -->

6. Conclusão
<!-- Síntese, próximos passos. Máx 1 parágrafo. -->

7. Referências
<!-- [1] Autor et al. (Ano). Título. Origem (A/B/C). DOI/URL -->
```

### 7.2 — Pandoc + Typst (PDF compilado)

```
templates/
├─ template.typ # Livro ABNT: capa, folha de rosto, CIP, sumário, 1.5
espacamento
├─ template_tcc.typ # TCC NBR 14724: 3cm margens, 1.5 espacamento, page
counter
├─ template_artigo.typ # Artigo NBR 6022: máx 20 pgs, 2 colunas,
abstract+keywords
└─ template_playbook.typ # Cards: ①②③④⑤⑥⑦ por card, grid layout
```

#### Exemplo: template.typ (Livro ABNT)

```
#set page(
 paper: "a4",
 margin: (top: 3cm, bottom: 2cm, left: 3cm, right: 2cm),
)

#set text(font: "Times New Roman", size: 12pt)
#set par(leading: 1.5em)

// Metadata (injetados por Pandoc via frontmatter YAML)
#let titulo = [#title]
```

```

#let autor = [#author]
#let anoPublicacao = [#date]

// Capa
#page(background: image("capa.png"))[
 // Ou renderizar programaticamente
 #align(center + horizon)[
 #text(size: 28pt, weight: "bold", fill: rgb("#1a1a1a"))[#titulo]
 #v(2em)
 #text(size: 14pt)[#autor]
]
]

// Folha de rosto
#page()[
 #text(size: 16pt, weight: "bold")[#titulo]
 #v(2em)
 #text(size: 12pt)[#autor]
 #v(1em)
 #text(size: 12pt)[
 Obra submetida como dissertação de mestrado...
]
]

// Conteúdo principal
#counter(page).update(1)
#body

```

**Adaptar:** substituir #title, #author, #body conforme Pandoc injetar.

## FASE 8: ESTRUTURA DE DADOS (JSON, estado imutável)

### 8.1 — config\_obra.json (criada em /esbocar, imutável)

```

{
 "slug": "livro-ia-agents-2026",
 "colecacao": "inteligencia-artificial",
 "tema": "Agentes de IA na Manufatura de Conteúdo",
 "tipo_obra": "livro",
 "natureza": "tecnica",
 "publico_alvo": "profissional",
 "nivel_tecnico": 6,
 "data_criacao": "2026-08-22",
 "data_limite": "2026-10-01",
 "gerar_campanha": true,
 "gerar_maquina": false,
 "capitulos_planejados": 18,
 "paginas_alvo": 300,
 "citacoes_esperadas": 200,
 "cta_url": "https://...",
 "cta_texto": "Adquira agora"
}

```

### 8.2 — estado\_fabrica.db (SQLite, MCP db\_state)

```

CREATE TABLE obras (
 slug TEXT PRIMARY KEY,
 tipo TEXT,
 colecacao TEXT,

```

```

 fase_atual TEXT, -- fase_1_pesquisa|fase_2_capitulos|fase_2_5_review|
 fase_3_compilacao|fase_4_colecao
 data_inicio TEXT,
 data_ultima_atualizacao TEXT,
 relatorio_auditoria JSON
);

```

```

CREATE TABLE capitulos (
 id TEXT PRIMARY KEY,
 slug TEXT,
 numero INT,
 titulo TEXT,
 status TEXT, -- rascunho|submetido|aceito|revisado|compilado
 metricas JSON, -- {palavras, citacoes, diagramas, linhas_codigo}
 data_criacao TEXT,
 data_conclusao TEXT,
 FOREIGN KEY (slug) REFERENCES obras(slug)
);

```

### 8.3 — Manifestos de Coleção (colecoes/.json)

```

{
 "nome": "inteligencia-artificial",
 "descricao": "Coleção: livro + artigo + e-book + playbook + lead magnet + deck + e-mails",
 "data_criacao": "2026-08-22",
 "data_ultima_atualizacao": "2026-08-23",
 "membros": [
 {
 "tipo": "livro",
 "slug": "livro-ia-agents-2026",
 "status": "compilado",
 "url_distribuicao": "output/inteligencia-artificial/livros/liv-2026-08-ia-agents/livro_final.pdf"
 },
 {
 "tipo": "artigo",
 "slug": "artigo-ia-agents-2026",
 "status": "em_revisao",
 "derivado_de": "livro-ia-agents-2026"
 }
],
 "campanhas": [
 {
 "material": "redes-sociais",
 "status": "agendada",
 "data_lancamento": "2026-10-15"
 }
],
 "maquinas": [
 {
 "slug_maquina": "maquina-vendas-ia-2026",
 "status": "pronta",
 "deployado_em": "https://vercel.com/meu-app"
 }
]
}

```

FASE 9: PORTABILIDADE MULTI-IDE (Junctions, Hardlinks, Scripts)

9.1 – Mapeamento de IDEs

IDE	Pasta	Link Necessário	Tipo
Claude Code	.claude/	(nativa)	—
Cursor	.cursor/rules/	junction → .claude/CLAUDE.md	SymbolicLink
OpenCode	.opencode/	junction → .claude/	SymbolicLink
Codebuff	.agents/	junction → .claude/{agents, commands}	SymbolicLink
VSCode	.vscode/ mcp.json	gerado por script	Generated

9.2 – setup-links.ps1 (Windows)

```
param([switch]$Verbose)

$ProjectRoot = $PSScriptRoot | Split-Path
$Claude = "$ProjectRoot\.claude"
$DotGit = "$ProjectRoot\.git"

Write-Host "🔗 Configurando portabilidade multi-IDE..."

Função auxiliar
function Create-Link($Target, $LinkPath, $Type = "SymbolicLink") {
 if (Test-Path $LinkPath) {
 Remove-Item $LinkPath -Force -ErrorAction SilentlyContinue
 }
 New-Item -ItemType $Type -Target $Target -Path $LinkPath -Force | Out-Null
 if ($Verbose) { Write-Host " ✓ $LinkPath → $Target" }
}

Junctions para portabilidade
Create-Link "$Claude" "$ProjectRoot/agentik"
Create-Link "$Claude/agents" "$ProjectRoot/.agents/agents"
Create-Link "$Claude/commands" "$ProjectRoot/.agents/commands"

Hardlink para CLAUDE.md (não é recopiado, apenas referenciado)
New-Item -ItemType HardLink -Path "$ProjectRoot/CLAUDE.md" -Target "$Claude/CLAUDE.md" -Force | Out-Null
if ($Verbose) { Write-Host " ✓ CLAUDE.md → hardlink" }

Recopiar hook (git hooks não suportam symlink seguro)
Copy-Item "$ProjectRoot/scripts/hooks/pre-commit" "$DotGit/hooks/pre-commit" -Force
if ($Verbose) { Write-Host " ✓ pre-commit hook copiado" }

Write-Host "✓ Portabilidade multi-IDE configurada"

Para Mac/Linux (setup-links.sh):

#!/bin/bash

PROJECT_ROOT="$(cd "$(dirname "$0")/.." && pwd)"
CLAUDE="$PROJECT_ROOT/.claude"
DOT_GIT="$PROJECT_ROOT/.git"
```

```

echo "🔗 Configurando portabilidade multi-IDE..."

ln -sf "$CLAUDE" "$PROJECT_ROOT/agentiC"
ln -sf "$CLAUDE/agents" "$PROJECT_ROOT/.agents/agents"
ln -sf "$CLAUDE/commands" "$PROJECT_ROOT/.agents/commands"
ln -sf "$CLAUDE/CLAUDE.md" "$PROJECT_ROOT/CLAUDE.md"
cp "$PROJECT_ROOT/scripts/hooks/pre-commit" "$DOT_GIT/hooks/pre-commit"
chmod +x "$DOT_GIT/hooks/pre-commit"

echo "✓ Portabilidade multi-IDE configurada"

```

## FASE 10: CI/CD & GATES (pré-commit, auditoria)

### 10.1 — Pre-commit Hook (scripts/hooks/pre-commit)

```

#!/bin/bash

set -e

1. Validação de sintaxe Python
echo "🔍 Validando scripts Python..."
python -m py_compile scripts/*.py
if [$? -ne 0]; then
 echo "❌ Syntax error in Python scripts"
 exit 1
fi

2. Auditoria de obra (se há output/ modificado)
CHANGED=$(git diff --cached --name-only | grep "^output/" | head -1 | cut -d/ -f2)
if [-n "$CHANGED"]; then
 echo "🔍 Auditando obra: $CHANGED"
 python scripts/auditar-obra.py "$CHANGED" --estrito
 if [$? -ne 0]; then
 echo "❌ Content audit failed for $CHANGED"
 exit 1
 fi
fi

3. Checklist mínima
if [! -f "CLAUDE.md"] || [! -f ".mcp.json"]; then
 echo "❌ CLAUDE.md ou .mcp.json faltando"
 exit 1
fi

echo "✓ Pre-commit checks passed"
exit 0

```

### 10.2 — Gates por Tipo de Obra

Em scripts/tipos\_obra.py:

```

"livro": {
 ...
 "gates_conteudo": [
 "validar-referencias", # URL/DOI reais
 "validar-metricas", # ≥1 métrica/cap
 "validar-escala", # contorno em "Aplica"
 "validar-afirmacoes", # dado factual com [N]
 "validar-fontes", # ≥70% A+B
]
}

```

```

 "validar-comandos-cli", # smoke test (opt-in)
],
}

```

Execução: `python scripts/auditar-obra.py <slug> --estrito` (encadeia todos em paralelo).

## FASE 11: TOKEN ECONOMY (estratégias de compressão)

### 11.1 — Skills de Economia

Skill	Aplicação	Economia
<b>caveman</b>	Pensar em telegrama (3–5 linhas); sem preâmbulos	<b>60%</b>
<b>headroom</b>	Logs/builds >7 linhas → 3 top + 4 bottom	<b>75%</b>
<b>lean-ctx</b>	grep antes de read; limitar linha	<b>40%</b>
<b>rtk</b>	Proxy bash (RTK Rust Token Killer) filtra output	<b>70%</b>
<b>pre-flight-check</b>	Validação pré-launch (evita retry loops)	<b>50%</b>

### 11.2 — Regras de Isenção

#### NUNCA COMPRIMIR:

- output/\*\*/\*.md, output/\*\*/\*.json (conteúdo de obra)
- auditar-obra.py, validar-codigo.py (gates determinísticas)
- revisor-tecnico relatorio (dados de qualidade)
- compilação Pandoc+Typst (obrigatória)

#### SEMPRE COMPRIMIR:

- Logs de build/test >7 linhas
- Output de comando verbose (git log, npm list, etc.)
- Transcrições longas de busca (grep, find)

#### USAR GRAFO (code-review-graph):

- Antes de leitura/busca (impacto analysis)
- Queries de dependência
- Verificação de cobertura

## FASE 12: DOCUMENTAÇÃO (templates de manual)

### 12.1 — Estrutura de Docs

#### docs/

```

├─ manual-completo-fabrica.md # Tudo em um lugar (~30 páginas)
├─ fluxo-fabrica-de-livros.md # Diagrama visual do fluxo
├─ fluxogramas-gauntlet-loop.md # P&D → Manufatura → Review → PDF
├─ plano-otimizacao-tokens.md # Como economizar contexto
├─ spec-aplicar-token-economy.md # Implementação técnica
├─ guia-execucao-maquina-vendas.md # Deploy + segurança (checklist obrigatória)
├─ playbook-campanhas-marketing.md # Moldes de campanha
├─ normas-abnt-referencia.md # Specs de formatação (NBR 14724, 6022)
├─ referencia-capa-design.md # 2D plano + ABNT + badge de nível
└─ manual-replicar-praticas-acima-media.md # Cultura da fábrica

```

## 12.2 — Relatório de Sessão (template)

# Relatório da Sessão: <tema>

**\*\*Data:\*\*** 2026-08-23 | **\*\*Operador:\*\*** Seu Nome | **\*\*Tempo Total:\*\*** HH:MM | **\*\*Modelo:\*\*** Claude Haiku 4.5

### ## Contexto

- **\*\*Obra:\*\*** `livro-ia-agents-2026` (slug)
- **\*\*Tipo:\*\*** Livro técnico
- **\*\*Fase:\*\*** 2 (Manufatura em Lotes)
- **\*\*Lote:\*\*** Capítulos 1-4

### ## Bugs Corrigidos (causa → fix)

#### ### 1. Grafo de referências quebrado (valores nulos em citação)

- **\*\*Causa:\*\*** `indexar-dossie.py` não tratava DOI vazio em entrada CSV
- **\*\*Fix:\*\*** Campo `doi` tornado opcional, fallback para URL; adicionado validação no parser
- **\*\*Teste:\*\*** `validar-referencias.py --sem-rede` passou; 52 refs validadas, 0 erros

#### ### 2. Codepage Windows rompendo relatórios

- **\*\*Causa:\*\*** `gerar-relatorio-sessao.py` printava `①②③` (Unicode) em console Windows cpl252
- **\*\*Fix:\*\*** Adicionado `console\_utf8()` a todos os scripts com emojis/símbolos
- **\*\*Teste:\*\*** Rodar em Windows PowerShell; relatório gerado sem erro

### ## Arquivos Alterados

.claude/skills/revisor-tecnico/SKILL.md | +80 linhas | Adição de checklist de peer review scripts/auditar-obra.py | ±45 linhas | Refator de gates, paralelo com Pool scripts/tipos\_obra.py | +15 linhas | Novo tipo: “revista” (experimental) docs/manual-completo-fabrica.md | +120 linhas | Sync de seção “Coleção V5” melhorias/2026-08-23-plano-replicacao-...md | 400 linhas | Este relatório (análise de replicação)

### ## Validações Rodadas

- ✅ **\*\*Python syntax:\*\*** `python -m py\_compile scripts/\*.py` (5/5 passed)
- ✅ **\*\*Auditoria:\*\*** `python scripts/auditar-obra.py livro-ia-agents-2026 --estrito` (6 gates passed)
- ✅ **\*\*Pre-commit hook:\*\*** `bash scripts/hooks/pre-commit` (passed)
- ✅ **\*\*Code-review-graph:\*\*** `code-review-graph status` (grafo atualizado)

### ## Commits Feitos

a1b2c3d feat(gates): adicionar validar-comandos-cli.py para smoke test CLI d4e5f6g fix(windows): console\_utf8() em todos scripts com print() h8i9j0k docs(manual): sync token economy strategies na seção 0 l1m2n3o chore(relatorio): salvar análise de replicação em melhorias/

### ## Próximos Passos

- [ ] Testar `/criar-livro <novo-tema>` com novo setup
- [ ] Validar junctions após clone em máquina nova
- [ ] Documentar padrão de extensão para novo tipo de obra
- [ ] Deploy de staging (máquina de vendas)

## ## Notas Operacionais

- **Token economy:** caveman + headroom + rtk economizou ~65% em contexto (4.2M → 1.5M)
- **Velocidade:** pool lote 4 (18 capítulos) completou em ~2h (paralelo)
- **Confiabilidade:** retry 3x + escalate para revisor capturou 100% de defeitos

## FASE 13: CHECKLIST FINAL DE REPLICAÇÃO

### 13.1 – Estrutura de Pastas (cópia direta)

```
novo-projeto/
├── .claude/
│ ├── CLAUDE.md (customizar tema/regras específicas)
│ ├── settings.json (adaptar paths absolutos)
│ ├── RTK.md (manter como-está)
│ ├── skills/ (copiar 50+ skills)
│ └── commands/ (copiar 19 commands, adaptar nomes se
novo tipo)
│ ├── agents/ (copiar 7 subagentes)
│ ├── mcp-servers/ (copiar 4 MCPs)
│ └── projects/novo-projeto/memory/ (novo: arquivo de auto-memória)
├── scripts/ (copiar 70+ scripts determinísticos)
├── templates/ (copiar Markdown/Typst/HTML)
├── docs/ (copiar + customizar para novo domínio)
├── melhorias/ (novo: registrar sessões e análises)
├── relatorios/ (novo: relatórios de sessão em MD+PDF)
├── output/ (novo: artefatos finais)
├── data/ (novo: estado_fabrica.db – SQLite)
├── CLAUDE.md (hardlink → .claude/CLAUDE.md)
├── .mcp.json (copiar, adaptar paths Windows)
├── setup-links.ps1 (script de portabilidade – rodar após
clone)
├── .git/
│ └── hooks/pre-commit (recopiar, não symlink)
```

### 13.2 – Passos de Configuração (pós-clone)

# 1. Criar junctions & links (Windows)

```
.\scripts\setup-links.ps1 -Verbose
```

# 2. Adaptar CLAUDE.md (customizar apenas regras de domínio)

# Não mudar: estrutura R1–R17, squad, fluxo

# Mudar: nomes de skills se novo domínio, tipos de obra, exemplos

# 3. Adaptar .mcp.json (paths absolutos)

# Substituir C:\Users\...\proj\_fabrica → C:\Users\...\novo-projeto

# 4. Inicializar banco de dados

```
python scripts/parametros_obra.py --criar-db
```

# 5. Teste estrutural (suíte completa)

```
python -m pytest scripts/ -q
```

```
code-review-graph status
```

# 6. Commit inicial

```
git add -A
```

```
git commit -m "chore(init): estrutura fábrica replicada, portabilidade configurada"
git push origin main
```

### 13.3 — Validação Pós-Replicação

#### [Governança]

- ✓ CLAUDE.md carregado (alwaysApply: true verificado)
- ✓ .mcp.json válido (MCPs iniciam sem erro)
- ✓ RTK.md presente (token economy configurada)
- ✓ Pre-commit hook funciona (bloqueia commit vermelho)

#### [Skills & Comandos]

- ✓ `/help` lista 50+ skills
- ✓ `/criar-livro`, `/criar-artigo`, `/criar-tcc` executáveis
- ✓ Skills de economia (caveman, headroom, lean-ctx) callable

#### [Subagentes & Scripts]

- ✓ Subagentes instanciáveis (pool-capitulos.py --plano lista lotes)
- ✓ Gates executam (auditar-obra.py --estrito sem erro)
- ✓ Scripts determinísticos (validar-\*.py, gerar-\*.py, compilar-\*.py)

#### [Portabilidade]

- ✓ `agentic/` → `.claude/`
- ✓ `.agents/{agents,commands}` → `.claude/{agents,commands}`
- ✓ `CLAUDE.md` hardlink
- ✓ `.git/hooks/pre-commit` copiado

#### [Infraestrutura]

- ✓ Code-review-graph sincronizado
- ✓ SQLite db\_state inicializado
- ✓ Junctions apontam corretamente (Mac: symlinks, Windows: junctions)
- ✓ Arquivo de memória criado (auto-memory)

## CONCLUSÃO

A **Fábrica Agêntica de Publicações V5** é um sistema robusto, modular e replicável. Cada novo projeto pode nascer com:

1.  Governança central (CLAUDE.md + settings.json + RTK.md)
2.  50+ skills especializadas (economia de tokens, redação, compilação)
3.  19 comandos orquestradores (fluxo imutável P&D → F3)
4.  7 subagentes paralelos (pesquisador, redators, revisor, ilustrador)
5.  70+ scripts determinísticos (gates, compilação, indexação)
6.  4 MCPs (estado, escrita, PDF, inteligência)
7.  Templates reutilizáveis (Markdown EITA-V2, Typst ABNT, HTML)
8.  V5 Coleções (cascata livro → derivadas)
9.  Portabilidade multi-IDE (Claude Code, Cursor, OpenCode, Codebuff, VSCode)
10.  Token economy ativa (até 70% de economia)

**Próximo passo:** usar este plano como blueprint para replicar em novos projetos, adaptando apenas:  
- Nome do projeto + domínio (CLAUDE.md) - Tipos de obra específicos (scripts/tipos\_obra.py) - Skills de domínio novo (se aplicável) - MCPs adicionais (conforme necessidade)

**Documento gerado:** 2026-08-23 | **Análise completa:** 13 fases, 30.000+ tokens | **Formato:** Markdown → PDF (Pandoc+Typst)